

Ensemble Learning

Technique in Machine learning where multiple models also called weak Learners are combined to form a stronger, accurate model.

Why?

- A Single model may **underfit** or **overfit**.
- Combining multiple models reduces bias, variance or both
- So, it improves accuracy, reduces overfitting, handling bias-variances trade-off and make the model more stable

Analogy : Taking Opinions from a panel of doctors instead of relying on one.

Types of Ensemble learning:

1. Bagging (Bootstrap Aggregating) :

- Training multiple models parallelly on random bootstrapped samples (replacement resampling) of the dataset.
- Combining results using majority voting (classification) or averaging (regression)
- Bagging reduce **variance**:

As we know that variance comes from **model sensitivity to training data**.

If we train a model (like a decision tree) on slightly different data, it can produce very different predictions → **high variance**.

Bagging tackles this by:

- Training **multiple models** on different samples of the data
- Then **averaging their predictions**

So, Errors from different models tend to **cancel out** when averaged

Example: Random Forest

Bagging Does not reduce bias

- Bias comes from **model assumptions** (e.g., linear model assuming linearity)
- Bagging does **not change the model type**
- It just trains the **same model** on different samples and averages

So the underlying assumption stays the same → **bias remains**

2. Boosting:

- Models are trained sequentially
- Each new model focuses on errors/residuals of the previous one or we can say that correcting the mistakes of the previous ones
- Instead of independent models (like bagging), boosting is **dependent learning**
- Boosting reduces Bias:

Step 1: Start with a simple (weak) model

- Example: a shallow decision tree (high bias, low variance)
- It makes systematic errors

Step 2: Identify errors

- Find where the model is wrong
- In some algorithms, misclassified points get **higher weight**

Step 3: Focus on mistakes

- Train the next model to **focus more on those difficult points**

Step 4: Combine models

- Add models together (not just average)
- Each new model **corrects previous bias**

Examples: Ada Boost (Adaptive Booting), Gradient Boosting, XGBoost.

3. Stacking:

(Stacked Generalization) is an ensemble learning technique where we combine

multiple different models and use another model to learn how to best combine their predictions.

Stacking learn how to combine models using another model Instead of Averaging (like bagging) or Sequential correction (like boosting)

How stacking works

Step 1: Train base models (Level-0 models)

we train multiple different models like:

- Decision Tree
- Logistic Regression
- SVM

Each model learns patterns differently

Step 2: Generate predictions

- Each model makes predictions on validation data
- These predictions become new features

Step 3: Train meta-model (Level-1 model)

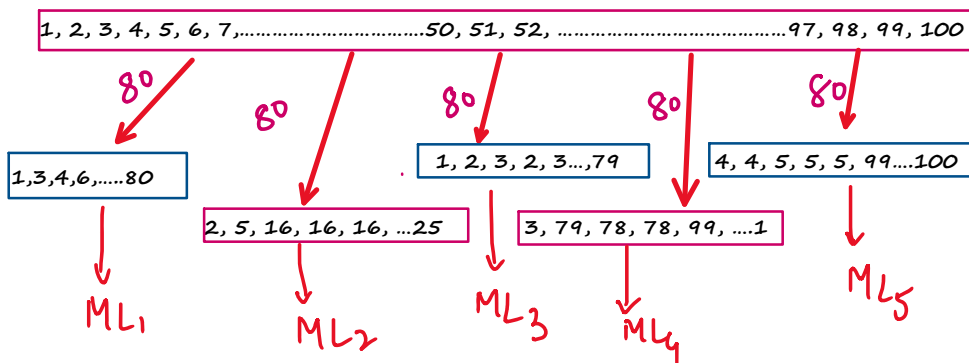
- A new model is trained on these predictions
- It learns which model to trust more in which situation

Analogy Multiple experts give opinions, A "judge" learns, Who is better in which case. So, Final decision is smarter than any single expert.

Important

- Bagging & Boosting can use different models
But in practice:
- Bagging → same model (trees)
- Boosting → same weak learners
- Stacking: Specifically designed to use diverse models

Bagging:



ML1, ML2, ML3, ML5, ML5, are model of same type, but these are trained on different subset of the dataset.

Bootstrap sampling

In bagging, we create multiple datasets using sampling with replacement

Each dataset:

- Has some repeated points
- Misses some original points
- Is slightly different

Problem without bootstrap

If we train multiple models on the same dataset

All models will be:

- Very similar

- Make similar errors

Averaging them won't help much

Bootstrap Aggregation (Bagging) is just a combination of two ideas:

Bootstrap + Aggregation

Bootstrap = Sampling with replacement

- From original dataset, we create multiple datasets
- Each dataset is: Same size, But contains repeated and missing points

Aggregation = Combining predictions

- Train a model on each bootstrap sample
- Then combine outputs: Regression → average
Classification → majority vote

Random Forest: A collection of multiple decision trees trained on random subsets of data and features, whose predictions are combined to give a final output.

Working:

1. Bootstrap Sampling (Bagging)

- Creating multiple datasets using sampling with replacement
- Each tree gets a different dataset

2. Training Decision Trees

- Training one tree on each dataset
- Trees are usually deep (high variance)

3. Random Feature Selection (VERY IMPORTANT)

- At each split, Only a random subset of features is considered, making trees even more different

4. Aggregation

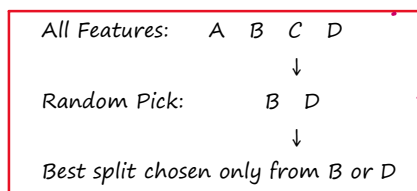
- Regression → average
- Classification → majority vote

So, randomness is introduced in TWO ways:

1. Random data (bootstrap)
2. Random features (feature subset selection)

This increases diversity → better performance

So, Random Forest = Bagging + Decision Trees + Random Feature Selection



Random Tree: is simply a decision tree built with randomness introduced during training. It is not a completely different algorithm, but a variation of a decision tree.

A random tree is just one decision tree with randomness (e.g., random features or random data). It is still a single model

In practice (especially in Random Forest), a random tree is built using:

1. Random data (Bootstrap)

- Each tree is trained on a random subset of data

2. Random feature selection

- At each split: Only a random subset of features is considered

So a “random tree” = Decision Tree + randomness in data and/or features

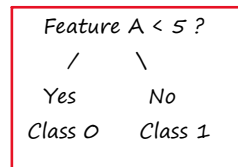
A random tree by itself is NOT an ensemble method.

Decision Stump:

A decision stump is a very simple decision tree with only one split

It has:

- One root node
- Two leaf nodes
- Depth = 1



Why random feature selection?

To make trees different from each other (reduce correlation)

If every tree: Uses all features, it always picks the best feature

Then: All trees look very similar, make similar mistakes, so, Averaging won't help much.

- With Random feature selection:

At each split: Only a random subset of features is available, So, Different trees pick different features, Trees become less similar

We can think it like:

- Feature A is very strong
- Without randomness → every tree uses A first

All trees become clones

key parameters of Random Forest

1. n_estimators (Number of trees)

- More trees → better performance (usually)
- But → slower computation

Normally used values: 100–500

2. max_features (Number of features considered at each split)

- 'sqrt' → $\sqrt{\text{total features}}$ (default for classification)
- 'log2' → $\log_2(\text{features})$
- Integer → fixed number

It Controls randomness

✓ Lower value → more randomness → less correlation

✓ Higher value → less randomness

3. max_depth

- High depth → complex tree → low bias, high variance
- Low depth → simple tree → high bias, low variance

✓ Controls overfitting

4. min_samples_split

Minimum samples required to split a node

- Higher value → fewer splits
- Prevents overfitting

5. min_samples_leaf

- A leaf node must contain at least X data points
- You don't allow splits that create very small groups
 - Larger value → smoother predictions
 - Reduces noise sensitivity

6. bootstrap

Whether to use bootstrap sampling

- True → Bagging (default)
- False → Use full dataset

7. criterion

How to measure split quality

- Classification:
 - "gini" (Gini impurity)
 - "entropy"
- Regression:
 - "squared_error"

8. random_state

Controls randomness

- Same value → reproducible results
- Different value → different trees

How many models (trees) to use?

In Random Forest, this is controlled by: n_estimators

- More trees → better performance (up to a point)
 - After some point → no significant improvement

How many subsets (bootstrap samples)?

In bagging / Random Forest:

✓ Number of subsets = number of trees

How many features to sample?

Controlled by: max_features

Rule of thumb

- Classification → $\sqrt{(\text{total features})}$
- Regression → $n_{\text{features}} / 3$

OOB (Out-of-Bag Samples)

In Random Forest:

- Each tree is trained on a bootstrap sample
- Some data points are not selected in that sample

Those unused data points = OOB samples

Original Data: [1 2 3 4 5]

Bootstrap Sample: [2 2 3 5 5]

OOB Samples: [1 4]

OOB acts like a validation set

For each data point:

- Use only trees where this point was OOB
- Get predictions
- Compare with actual value

Compute OOB error

If we are using OOB then:

- ✓ No need for separate validation set
- ✓ Gives reliable performance estimate

OOB working

In Random Forest:

Step 1: Train trees

- Suppose 100 data points
- For each tree:
 - Train on ~63 points (bootstrap sample)
 - ~37 points are OOB for that tree

Step 2: For ONE data point (important)

Let's take Data Point #10

It will be:

- Used in some trees
- OOB (unused) in some trees

Step 3: Prediction for that data point

We DO NOT use all trees

We use only trees where this point was OOB

For each data point:

Final prediction = Average (or vote) of predictions from trees where it was OOB

Tree 1 → used → ignore
Tree 2 → OOB → use prediction
Tree 3 → OOB → use prediction
Tree 4 → used → ignore

Training phase:

- Build trees using bootstrap

Validation phase (inside training):

- Use OOB predictions → compute OOB error

Final testing:

- Use completely separate test set

Analogy

- Train (Learning) → Studying
- Validation (Evaluation & tuning) → Practice tests
- Test (Final Evaluation) → Final exam

Why not use predictions from trees where the data point WAS used?"

---Because those trees have already seen that data point

So:

- The tree may have memorized it
- Prediction will be too optimistic

This is called *overfitting bias*

◊ *Example (very important)*

Suppose:

- True value = 10

Tree trained on that point:

- Predicts = 10 (perfect)

But Not reliable (it already saw it)

Tree that did NOT see it (OOB):

- Predicts = 8 or 9

More realistic estimate

We only trust predictions from models that: Have not seen the data point during training

Analogy

- Student sees question before exam → scores 100 (not fair)
- Student sees question first time → real ability